BNN_FCC Contest Submission Report

Khan Thamid Hasan
UFID: 17308681
khanthamidhasan@ufl.edu

April 17, 2026

Repository root:	`bnn_fcc_contest`
Top-level DUT:	`rtl/bnn_fcc.sv`
Timing target:	`XCU250-FIGD2104-2L-E`
Canonical candidate:	`PARALLEL_INPUTS = 8, PARALLEL_NEURONS = {8,8,1}`

Executive Summary

Item	Result
Targeted use case	High-frequency, BRAM-backed implementation with conservative output-layer parallelism
Required topology	MNIST SFC: `784 -> 256 -> 256 -> 10`
Canonical submission parameters	`PARALLEL_INPUTS = 8`, `PARALLEL_NEURONS = {8,8,1}`
Openflex timing flow	PASS
Measured f_{max}	**400.48 MHz**
LUTs	**3066**
FFs	**1818**
BRAM	**12.5**
DSP	**0**
Additional coverage testbench	PASS, average covergroup score **96.1%**
Performance-oriented simulation run	PASS, with reported results from `sim/perf_out.txt` listed in Section 6.3

Submission note. During final repo cleanup, the default `PARALLEL_NEURONS` setting in `verification/bnn_fcc_t` was synchronized with the DUT and timing wrapper so that the design being documented, verified, and timed is the same candidate. If you regenerate logs after that synchronization, update the small numerical tables in Sections 6.2 and 6.3 before producing the final PDF.

1 Design Objective and Tradeoff

This submission targets a believable high-frequency operating point on the provided UltraScale+ target rather than maximum raw output-layer spatial parallelism. The main architectural tradeoff is to keep the hidden layers moderately parallel while serializing the small output layer:

- `PARALLEL_INPUTS = 8`

- Hidden-layer parallelism: `PARALLEL_NEURONS[0] = 8`, `PARALLEL_NEURONS[1] = 8`
- Output-layer parallelism: `PARALLEL_NEURONS[2] = 1`

The intent is to reduce routed timing pressure on the BRAM → XNOR → popcount → accumulate path while keeping memory organization simple and timing-friendly. This approach favors a stronger timing result and a smaller, cleaner hardware footprint at the cost of some end-to-end throughput relative to a more spatially aggressive output layer.

2 Repository Structure

Path	Purpose
`rtl/`	Top-level DUT and interface specification
`verification/`	Required functional testbench plus additional coverage-oriented testbench
`openflex/`	Openflex YAML flows, timing wrapper, helper parser patch, and flow notes
`python/`	Model data and reference test vectors used by the testbench infrastructure
`doc/`	Background slides and supporting material
`report.tex`	Canonical source for the submission report

3 How to Run the Repository

3.1 Prerequisites

- Siemens Questa/ModelSim for simulation
- Xilinx Vivado for synthesis, place, and route
- Python virtual environment containing `openflex`
- Linux shell environment for the contest flows

3.2 Build a local simulator work library

From the repository root:

```
mkdir -p sim
cd sim
vlib work

vlog -sv ../rtl/bnn_fcc.sv \
         ../verification/axi4_stream_if.sv \
         ../verification/bnn_fcc_tb_pkg.sv \
         ../verification/bnn_fcc_tb.sv \
         ../verification/bnn_fcc_coverage_tb.sv \
         ../openflex/rtl/bnn_fcc_timing.sv
```

3.3 Required functional simulation

This is the direct simulation of the provided top-level testbench:

```
vsim -c work.bnn_fcc_tb -do "run -all; quit -f" | tee tb_out.txt
grep -E "SUCCESS:|FAILED:|Avg latency|Avg throughput" tb_out.txt
```

3.4 Performance-oriented simulation

To measure performance without external throttling, use the measured timing result as the clock period and disable randomized source/sink stalls:

```
vsim -c work.bnn_fcc_tb \
  -gCLK_PERIOD=2.497ns \
  -gTOGGLE_DATA_OUT_READY=0 \
  -gCONFIG_VALID_PROBABILITY=1.0 \
  -gDATA_IN_VALID_PROBABILITY=1.0 \
  -do "run -all; quit -f" | tee perf_out.txt

grep -E "SUCCESS:|Avg latency|Avg throughput" perf_out.txt
```

3.5 Coverage-oriented simulation

The repository includes an additional self-checking coverage testbench:

```
vsim -c work.bnn_fcc_coverage_tb -do "run -all; quit -f" | tee coverage_tb_out.txt
tail -n 40 coverage_tb_out.txt
```

3.6 Openflex verification and timing flows

From the repository root:

```
cd openflex
source ~/envs/openflex/bin/activate
```

If openflex crashes while parsing a completed Vivado run because Vivado reports fractional utilization values such as `12.5` BRAMs, patch the installed parser:

```
python patch_openflex_parser.py --check
python patch_openflex_parser.py
```

Then run the canonical contest flows:

```
openflex bnn_fcc_verify.yml | tee verify_out.txt
openflex bnn_fcc_coverage.yml | tee coverage_out.txt
openflex bnn_fcc_timing.yml -c bnn_fcc.csv | tee timing_out.txt
```

Useful summaries:

```
grep -E "SUCCESS:|FAILED:|FAILURE:" verify_out.txt
grep -E "WNS|Post Physical Optimization Timing Summary|8-7052|8-7030|8-6904" timing_out.
    txt
cat bnn_fcc.csv
```

3.7 Build the submission PDF

From the repository root:

```
latexmk -pdf report.tex
```

If `latexmk` is unavailable:

```
pdflatex report.tex
pdflatex report.tex
```

4 Architecture Summary

4.1 Top-level functionality

The DUT implements a parameterized fully connected binary neural network classifier with three AXI4-Stream interfaces:

- Configuration input for weights and thresholds
- Image input stream for 8-bit pixels
- Output stream for the predicted class index

The reference contest topology is the small fully connected FINN-style network:

`784 inputs -> 256 hidden -> 256 hidden -> 10 outputs`

4.2 Key implementation choices

- The configuration stream is parsed internally from the provided message headers rather than assuming a single fixed message order.
- Image pixels are binarized against `8'd128` to match the provided verification model.
- Weights are stored in packed words with width `PARALLEL_INPUTS = 8`, enabling BRAM-backed storage that matches the datapath width.
- Threshold memories for the hidden layers are inferred as LUTRAM, which Vivado reports as the efficient choice for the threshold array sizes in this design.
- Inference proceeds layer by layer, with hidden layers producing binarized activations and the output layer performing argmax over population counts.
- Output backpressure is honored by holding a valid result until `data_out_ready` is asserted.

4.3 Timing-oriented refinements

The latest submission candidate includes several low-risk timing improvements:

- The popcount across the 8-bit XNOR word is implemented as a fixed balanced tree rather than a generic loop-based reduction.

- The output layer is serialized by choosing `PARALLEL_NEURONS[2] = 1`, which reduces pressure on the routed critical path.
- The BRAM weight bank logic was restructured to present a cleaner read/write organization to synthesis.
- The hidden-layer parallelism remains matched to `PARALLEL_INPUTS` to preserve straightforward packing and inter-layer buffering.

5 Verification Strategy

5.1 Required contest testbench

The required functional verification infrastructure is the provided `verification/bnn_fcc_tb.sv`. It checks:

- Correct parsing and loading of the trained model
- Correct MNIST classification outputs
- Robustness to randomized `config_valid` gaps
- Robustness to randomized `data_in_valid` gaps
- Robustness to randomized `data_out_ready` backpressure
- Reported latency and throughput statistics

5.2 Additional coverage-oriented testbench

The repository also includes `verification/bnn_fcc_coverage_tb.sv`, which was added specifically to improve verification quality beyond the basic contest pass/fail flow. This testbench stresses:

- Reordered configuration messages
- Partial `TKEEP` on both configuration and image traffic
- Reset during configuration and reset during image streaming
- Continuous, random, and bursty valid / ready behavior
- Repeated outputs versus varying outputs
- Coverage of all output classes in a self-checking randomized scenario

The coverage testbench prints both a success banner and a per-covergroup summary, which makes it easy for an evaluator to inspect coverage quality without digging into simulator databases.

6 Results

6.1 Timing and resource utilization

The measured timing and area results from `openflex/bnn_fcc.csv` are summarized below.

Metric	Value
f_{max}	400.48057669203047 MHz
LUT	3066
LUT as logic	2298
LUT as memory	768
FF / registers	1818
BRAM	12.5
DSP	0

Vivado also reports the expected memory implementation split for this architecture:

- Weight banks inferred into BRAM
- Threshold memories inferred into LUTRAM because BRAM would be inefficient for the small threshold arrays
- Timing warnings indicating that optional BRAM output registers were not merged, which is consistent with the remaining critical path pressure reported during implementation

6.2 Verification and coverage results

The additional coverage-oriented run reported the following summary:

Covergroup / check	Result
`config_msg_cov`	95.8%
`keep_cov`	85.4%
`valid_mode_cov`	100.0%
`ready_mode_cov`	100.0%
`reset_cov`	100.0%
`output_cov`	100.0%
`output_seq_cov`	100.0%
`threshold_cov`	94.4%
`weight_cov`	88.9%
Average covergroup score	**96.1%**
Pass / fail banner	`SUCCESS: bnn_fcc_coverage_tb completed with 16 passing checks.`

6.3 Performance-oriented run

The latest supplied direct performance run from `sim/perf_out.txt` reported:

Metric	Value
Pass / fail banner	`SUCCESS: all 50 tests completed successfully.`
Average latency	89694.8 cycles / image
Average latency (time)	223968.0 ns / image
Average throughput	4464.5 outputs / sec
Average cycles / output	89704.3

Important cleanup note. Because the required testbench defaults were synchronized during final repository cleanup, this performance table should be refreshed after rerunning the command in Section 3.4 if you want every reported number in the PDF to correspond exactly to the final repository state.

6.4 Representative console snapshot

Figure 1 shows a representative terminal capture from the final verification workflow. The image includes the passing coverage summary and the performance-oriented testbench statistics that were used when preparing this submission package.

7 Why This Is a Reasonable Contest Candidate

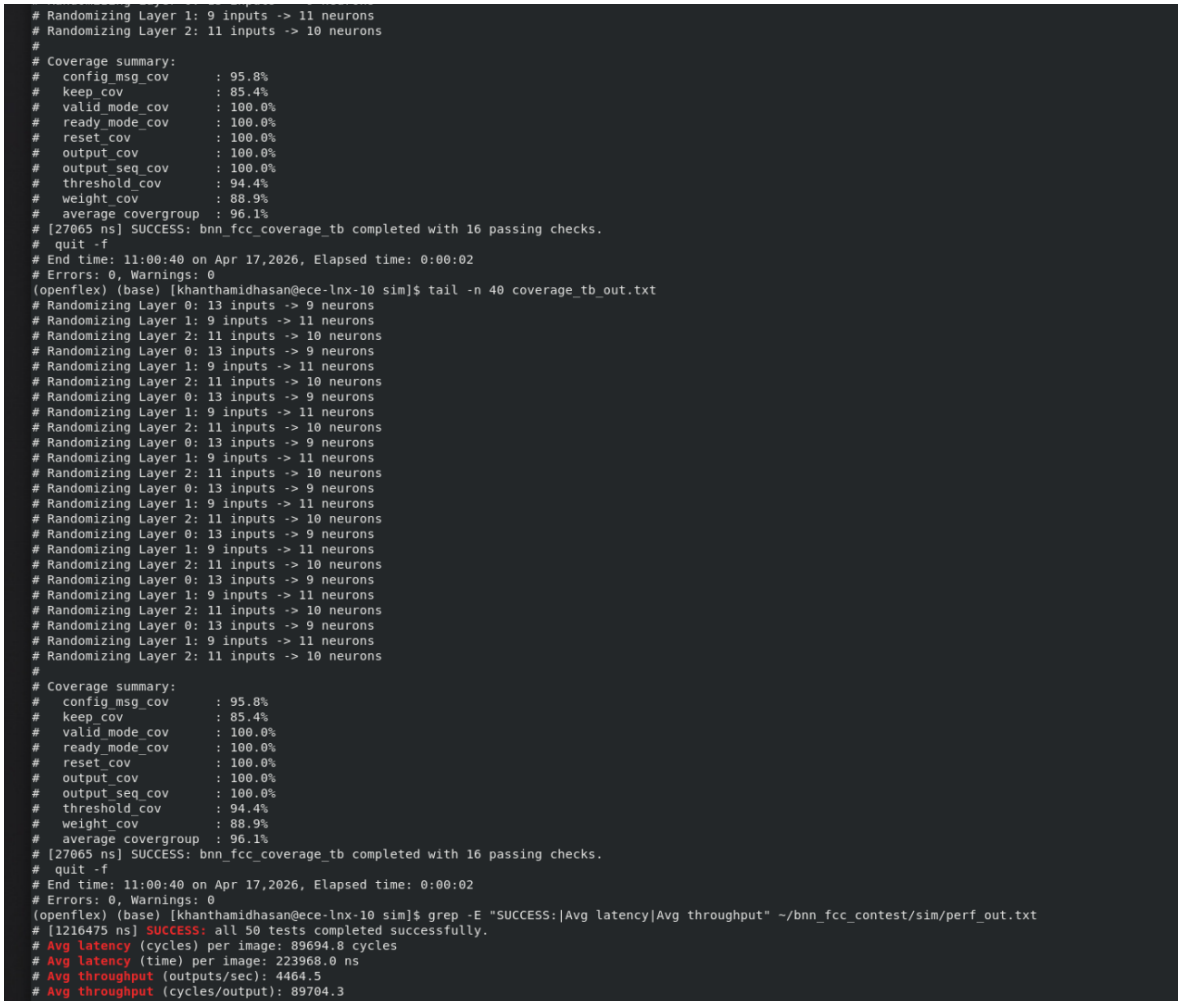
This submission is credible for the contest because it makes a coherent and defensible tradeoff:

- It does not rely on unrealistic outlier timing claims. The measured f_{max} is in a believable range for a BRAM-backed UltraScale+ design.
- It keeps resource use small: only 3066 LUTs, 1818 FFs, 12.5 BRAMs, and 0 DSPs.
- It includes extra verification infrastructure rather than only the minimum required contest testbench.
- The repo contains the exact files needed to rerun functional verification, coverage, and timing from the command line.

8 Submission Checklist

Before turning in the repository, the following should be true:

1. `verification/bnn_fcc_tb.sv`, `verification/bnn_fcc_coverage_tb.sv`, `rtl/bnn_fcc.sv`, and `openflex/rtl/bnn_fcc_timing.sv` all use the same canonical submission parameters.
2. The required functional simulation passes with no failing tests.
3. The additional coverage-oriented simulation passes and prints the coverage summary.
4. `openflex bnn_fcc_verify.yml` passes.
5. `openflex bnn_fcc_timing.yml -c bnn_fcc.csv` completes and reports the expected timing / area values.
6. `report.pdf` is rebuilt from this `report.tex` source after the final verification and timing reruns.



```
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
#
# Coverage summary:
#   config_msg_cov      : 95.8%
#   keep_cov            : 85.4%
#   valid_mode_cov      : 100.0%
#   ready_mode_cov      : 100.0%
#   reset_cov           : 100.0%
#   output_cov          : 100.0%
#   output_seq_cov      : 100.0%
#   threshold_cov       : 94.4%
#   weight_cov          : 88.9%
#   average covergroup  : 96.1%
# [27065 ns] SUCCESS: bnn_fcc_coverage_tb completed with 16 passing checks.
#  quit -f
# End time: 11:00:40 on Apr 17,2026, Elapsed time: 0:00:02
# Errors: 0, Warnings: 0
(openflex) (base) [khanthamidhasan@ece-lnx-10 sim]$ tail -n 40 coverage_tb_out.txt
# Randomizing Layer 0: 13 inputs -> 9 neurons
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
# Randomizing Layer 0: 13 inputs -> 9 neurons
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
# Randomizing Layer 0: 13 inputs -> 9 neurons
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
# Randomizing Layer 0: 13 inputs -> 9 neurons
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
# Randomizing Layer 0: 13 inputs -> 9 neurons
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
# Randomizing Layer 0: 13 inputs -> 9 neurons
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
# Randomizing Layer 0: 13 inputs -> 9 neurons
# Randomizing Layer 1: 9 inputs -> 11 neurons
# Randomizing Layer 2: 11 inputs -> 10 neurons
#
# Coverage summary:
#   config_msg_cov      : 95.8%
#   keep_cov            : 85.4%
#   valid_mode_cov      : 100.0%
#   ready_mode_cov      : 100.0%
#   reset_cov           : 100.0%
#   output_cov          : 100.0%
#   output_seq_cov      : 100.0%
#   threshold_cov       : 94.4%
#   weight_cov          : 88.9%
#   average covergroup  : 96.1%
# [27065 ns] SUCCESS: bnn_fcc_coverage_tb completed with 16 passing checks.
#  quit -f
# End time: 11:00:40 on Apr 17,2026, Elapsed time: 0:00:02
# Errors: 0, Warnings: 0
(openflex) (base) [khanthamidhasan@ece-lnx-10 sim]$ grep -E "SUCCESS:|Avg latency|Avg throughput" ~/bnn_fcc_contest/sim/perf_out.txt
# [1216475 ns] SUCCESS: all 50 tests completed successfully.
# Avg latency (cycles) per image: 89694.8 cycles
# Avg latency (time) per image: 223968.0 ns
# Avg throughput (outputs/sec): 4464.5
# Avg throughput (cycles/output): 89704.3
```

Figure 1: Representative terminal snapshot showing the final coverage summary and performance-oriented simulation results.

Appendix: Exact Commands

```
# compile
mkdir -p sim
cd sim
vlib work
vlog -sv ../rtl/bnn_fcc.sv \
         ../verification/axi4_stream_if.sv \
         ../verification/bnn_fcc_tb_pkg.sv \
         ../verification/bnn_fcc_tb.sv \
         ../verification/bnn_fcc_coverage_tb.sv \
         ../openflex/rtl/bnn_fcc_timing.sv

# required simulation
vsim -c work.bnn_fcc_tb -do "run -all; quit -f" | tee tb_out.txt

# performance simulation
vsim -c work.bnn_fcc_tb \
  -gCLK_PERIOD=2.497ns \
  -gTOGGLE_DATA_OUT_READY=0 \
  -gCONFIG_VALID_PROBABILITY=1.0 \
  -gDATA_IN_VALID_PROBABILITY=1.0 \
  -do "run -all; quit -f" | tee perf_out.txt

# coverage simulation
vsim -c work.bnn_fcc_coverage_tb -do "run -all; quit -f" | tee coverage_tb_out.txt

# openflex
cd ../openflex
source ~/envs/openflex/bin/activate
python patch_openflex_parser.py --check
python patch_openflex_parser.py
openflex bnn_fcc_verify.yml | tee verify_out.txt
openflex bnn_fcc_coverage.yml | tee coverage_out.txt
openflex bnn_fcc_timing.yml -c bnn_fcc.csv | tee timing_out.txt

# build report
cd ..
latexmk -pdf report.tex
```